

PRACTICAL RECORD

DBMS Lab

Course Code: U25AMILPC307

PRACTICAL P1

Database Creation and ER Modeling – Designing
an ER diagram for a real-world case study and
mapping it into a relational schema

Case Study: **College Management System**

STUDENT DETAILS

Name	Sagar Kumar
PRN	SOE25BTAM29
Section	A
Program	Bachelor of Technology
Year	2
Semester	III
Course Code	U25AMILPC307
Course Title	DBMS Lab
Practical	P1

1. Experiment Number and Title

Experiment No.	P1
Title	Database Creation and ER Modeling – Designing an ER diagram for a real-world case study and mapping it into a relational schema
Case Study	College Management System
Software Used	PostgreSQL 17 (server and psql client)

2. Aim

To design an Entity–Relationship (ER) model for a College Management System, map the ER model into a relational schema, and create the corresponding database and tables in PostgreSQL 17 with appropriate primary key, foreign key and integrity constraints.

3. Problem Statement

A college needs to store and manage information about its departments, students, faculty members, the courses offered, and the enrollment of students in those courses. Every student and every faculty member belongs to exactly one department, every course is taught by one faculty member, and a student may enroll in many courses while a course may have many students. For each enrollment, the date of enrollment and the marks obtained must be recorded.

Design an ER diagram that captures these requirements, convert it into a relational schema, and implement the schema as a PostgreSQL database named `college_db`.

4. Objectives

1. To identify the entities, attributes and relationships of a real-world system.
2. To select a primary key for every entity and identify the foreign keys needed to link entities.
3. To determine the cardinality (1:N, M:N) and participation of each relationship.
4. To draw the ER diagram using standard (Chen) notation.
5. To map the ER diagram into a relational schema using standard ER-to-relational mapping rules.
6. To create the database and tables in PostgreSQL 17 using DDL commands and verify them.

5. Case Study Description – College Management System

The college is organised into **departments** (for example, Computer Science and Engineering, Artificial Intelligence and Machine Learning). Each department is identified by a department ID and has a unique name.

Students are admitted into a department. For every student the college records a student ID, name, e-mail address (unique for each student) and phone number. **Faculty** members are also attached to a single

department and are identified by a faculty ID.

The college offers **courses**. Each course has a course ID, a name and a number of credits, and is assigned to one faculty member who teaches it; a faculty member may teach several courses.

Students register for courses. One student can take many courses and one course can be taken by many students, so this is a many-to-many association. Each registration is stored as an **enrollment** record holding the enrollment ID, the student, the course, the enrollment date and the marks scored (marks may be empty until the course is evaluated). A student cannot enroll in the same course twice.

6. Entities

#	Entity	Description
1	Department	An academic department of the college.
2	Student	A student admitted to a department.
3	Faculty	A teaching staff member working in a department.
4	Course	A subject offered by the college and taught by a faculty member.
5	Enrollment	Associative entity that records a student's registration in a course (resolves the M:N relationship between Student and Course).

7. Attributes

Entity	Attribute	Data Type (PostgreSQL)	Constraint / Description
Department	department_id	INT (identity)	Primary key, auto-generated
	department_name	VARCHAR(100)	NOT NULL, UNIQUE
Student	student_id	INT (identity)	Primary key, auto-generated
	student_name	VARCHAR(100)	NOT NULL
	email	VARCHAR(120)	NOT NULL, UNIQUE
	phone	VARCHAR(15)	Optional contact number
	department_id	INT	NOT NULL, foreign key → Department
Faculty	faculty_id	INT (identity)	Primary key, auto-generated
	faculty_name	VARCHAR(100)	NOT NULL
	department_id	INT	NOT NULL, foreign key → Department

Course	course_id	INT (identity)	Primary key, auto-generated
	course_name	VARCHAR(100)	NOT NULL
	credits	SMALLINT	NOT NULL, CHECK 1 to 6
	faculty_id	INT	NOT NULL, foreign key → Faculty
Enrollment	enrollment_id	INT (identity)	Primary key, auto-generated
	student_id	INT	NOT NULL, foreign key → Student
	course_id	INT	NOT NULL, foreign key → Course
	enrollment_date	DATE	NOT NULL, DEFAULT CURRENT_DATE
	marks	NUMERIC(5,2)	Optional, CHECK 0 to 100

All attributes are *simple* and *single-valued*. The pair (student_id, course_id) in Enrollment is additionally declared UNIQUE so that a student cannot enroll in the same course twice.

8. Primary Keys

Table	Primary Key	Constraint Name	Reason
department	<u>department_id</u>	pk_department	Uniquely identifies each department.
student	<u>student_id</u>	pk_student	Uniquely identifies each student.
faculty	<u>faculty_id</u>	pk_faculty	Uniquely identifies each faculty member.
course	<u>course_id</u>	pk_course	Uniquely identifies each course.
enrollment	<u>enrollment_id</u>	pk_enrollment	Surrogate key for each enrollment; (student_id, course_id) is kept as an alternate (candidate) key using UNIQUE.

9. Foreign Keys

Table	Foreign Key	References	Constraint Name	On Delete
student	department_id	department(department_id)	fk_student_department	RESTRICT
faculty	department_id	department(department_id)	fk_faculty_department	RESTRICT
course	faculty_id	faculty(faculty_id)	fk_course_faculty	RESTRICT
enrollment	student_id	student(student_id)	fk_enrollment_student	CASCADE

enrollment	<i>course_id</i>	course(course_id)	fk_enrollment_course	CASCADE
------------	------------------	-------------------	----------------------	---------

All foreign keys use ON UPDATE CASCADE. A department or faculty member that is still referenced cannot be deleted (RESTRICT), whereas deleting a student or a course automatically removes the related enrollment rows (CASCADE).

10. Relationships and Cardinalities

Relationship	Entities	Cardinality	Meaning and Participation
Belongs_To	Department – Student	1 : N	One department has many students; each student belongs to exactly one department. Student participation is total (department_id NOT NULL); Department participation is partial.
Works_In	Department – Faculty	1 : N	One department has many faculty members; each faculty member works in exactly one department. Faculty participation is total .
Teaches	Faculty – Course	1 : N	One faculty member teaches many courses; each course is taught by exactly one faculty member. Course participation is total .
Enrolls (via Enrollment)	Student – Course	M : N	A student enrolls in many courses and a course has many students. Implemented through the Enrollment table as two 1 : N relationships: Student 1 : N Enrollment and Course 1 : N Enrollment. Participation of both sides is partial.

11. ER Diagram

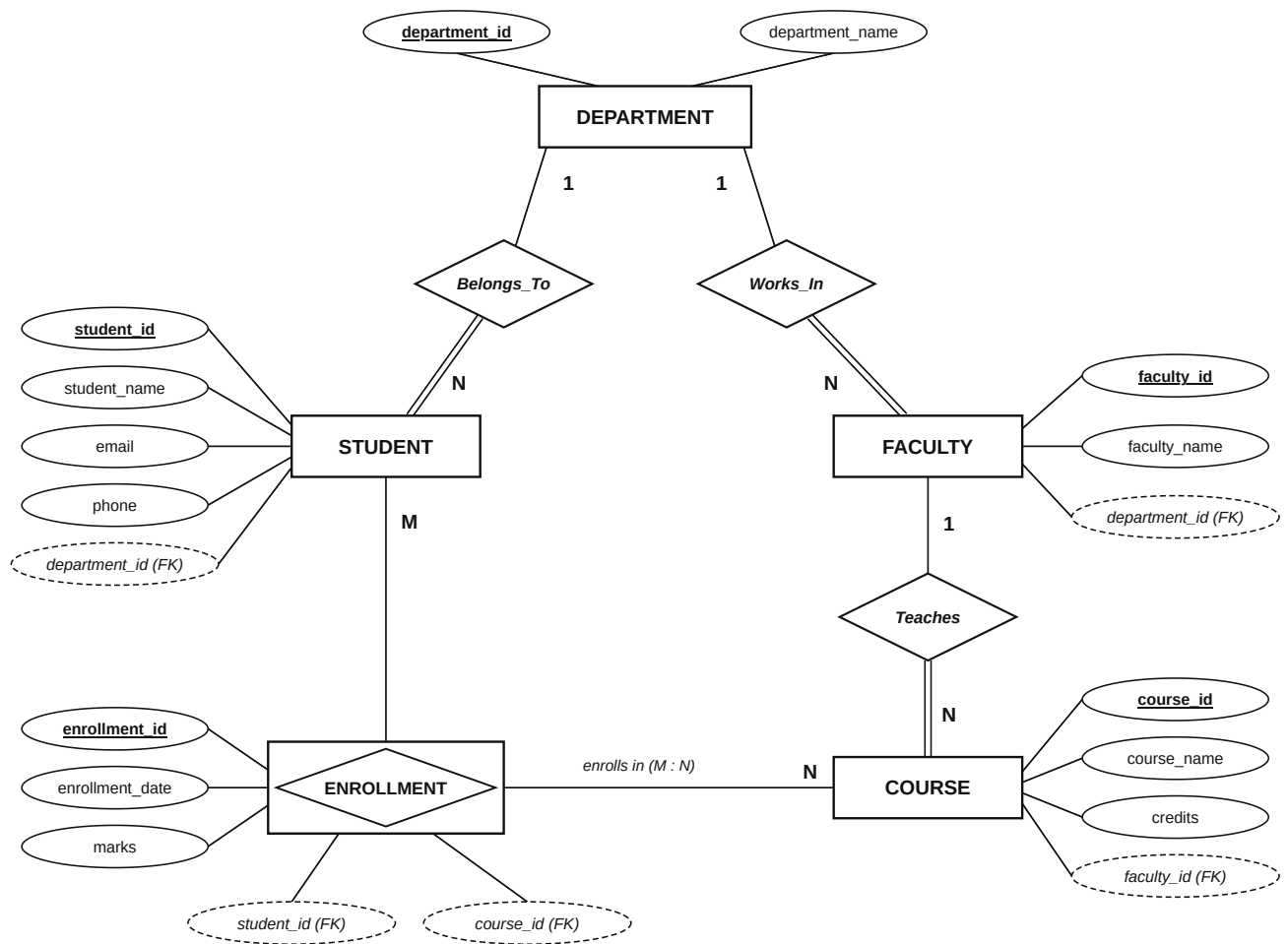


Figure 1: ER diagram of the College Management System (Chen notation)

Notation used in Figure 1	
Rectangle	Entity
Rectangle enclosing a diamond	Associative entity (Enrollment, resolving Student M : N Course)
Diamond	Relationship
Ellipse with <u>underlined</u> name	Primary key attribute
Dashed ellipse marked (FK)	Foreign key attribute (shown to make the mapping explicit)
1, N, M on a line	Cardinality of that side of the relationship
Double line	Total participation of the entity in the relationship

12. Relational Schema

12.1 Mapping Rules Applied

1. **Strong entities:** Department, Student, Faculty and Course each become a table containing their simple attributes; the key attribute becomes the PRIMARY KEY.
2. **1 : N relationships** (Belongs_To, Works_In, Teaches): the primary key of the “1” side is added as a FOREIGN KEY in the table of the “N” side – department_id in student and faculty, faculty_id in course. Because participation of the N side is total, these columns are NOT NULL.
3. **M : N relationship** (Student–Course): a separate table *enrollment* is created. It contains the primary keys of both participating entities as foreign keys (student_id, course_id) and the attributes of the relationship (enrollment_date, marks). A surrogate primary key enrollment_id is used, and UNIQUE (student_id, course_id) preserves the rule that a student enrolls in a course only once.

12.2 Schema (PK underlined, FK in italics)

DEPARTMENT (department_id, department_name)

STUDENT (student_id, student_name, email, phone, *department_id* → DEPARTMENT)

FACULTY (faculty_id, faculty_name, *department_id* → DEPARTMENT)

COURSE (course_id, course_name, credits, *faculty_id* → FACULTY)

ENROLLMENT (enrollment_id, *student_id* → STUDENT, *course_id* → COURSE, enrollment_date, marks)

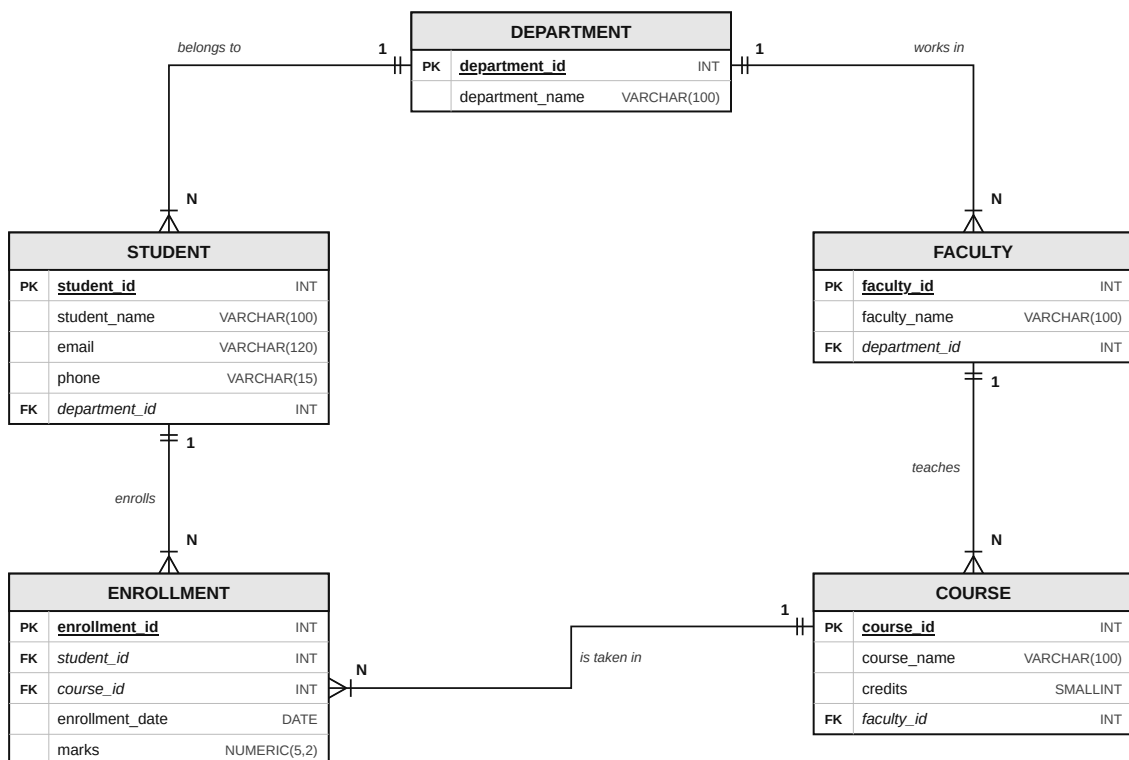


Figure 2: Relational schema diagram showing tables, PK/FK columns and 1 : N links (crow's-foot marks the “many” side)

13. Explanation of the ER Model

- **Department** is the central entity. It takes part in two one-to-many relationships: *Belongs_To* with Student and *Works_In* with Faculty. A department may exist before any students or faculty are added (partial participation), but every student and every faculty member must belong to a department (total participation).
- **Faculty** and **Course** are linked by *Teaches* (1 : N). Each course has exactly one faculty member in charge, so `faculty_id` is stored in the course table and is NOT NULL.
- **Student** and **Course** have a many-to-many relationship. A relational table cannot store a list of courses in a single student row, so the relationship is modelled as the associative entity **Enrollment**. Each enrollment row links one student to one course and also holds the facts that belong to that pairing – the enrollment date and the marks.
- Keys enforce correctness: primary keys prevent duplicate rows, foreign keys maintain *referential integrity* (an enrollment cannot point to a student who does not exist), UNIQUE prevents duplicate e-mail addresses, department names and repeat enrollments, and CHECK constraints keep credits between 1 and 6 and marks between 0 and 100.
- Identity columns (`GENERATED ALWAYS AS IDENTITY`, the SQL-standard replacement for SERIAL in PostgreSQL) generate key values automatically.

14. SQL Commands for Database Creation (PostgreSQL 17)

The commands below create the database college_db and its five tables. Parent tables are created before the tables that reference them.

```
-- Create the database and connect to it
CREATE DATABASE college_db;
\c college_db

-- Parent table
CREATE TABLE department (
    department_id INT GENERATED ALWAYS AS IDENTITY,
    department_name VARCHAR(100) NOT NULL,
    CONSTRAINT pk_department PRIMARY KEY (department_id),
    CONSTRAINT uq_department_name UNIQUE (department_name)
);

-- Department 1 : N Student
CREATE TABLE student (
    student_id INT GENERATED ALWAYS AS IDENTITY,
    student_name VARCHAR(100) NOT NULL,
    email VARCHAR(120) NOT NULL,
    phone VARCHAR(15),
    department_id INT NOT NULL,
    CONSTRAINT pk_student PRIMARY KEY (student_id),
    CONSTRAINT uq_student_email UNIQUE (email),
    CONSTRAINT fk_student_department FOREIGN KEY (department_id)
        REFERENCES department (department_id)
        ON UPDATE CASCADE ON DELETE RESTRICT
);

-- Department 1 : N Faculty
CREATE TABLE faculty (
    faculty_id INT GENERATED ALWAYS AS IDENTITY,
    faculty_name VARCHAR(100) NOT NULL,
    department_id INT NOT NULL,
    CONSTRAINT pk_faculty PRIMARY KEY (faculty_id),
    CONSTRAINT fk_faculty_department FOREIGN KEY (department_id)
        REFERENCES department (department_id)
        ON UPDATE CASCADE ON DELETE RESTRICT
);

-- Faculty 1 : N Course
CREATE TABLE course (
    course_id INT GENERATED ALWAYS AS IDENTITY,
    course_name VARCHAR(100) NOT NULL,
    credits SMALLINT NOT NULL,
    faculty_id INT NOT NULL,
    CONSTRAINT pk_course PRIMARY KEY (course_id),
    CONSTRAINT chk_course_credits CHECK (credits BETWEEN 1 AND 6),
    CONSTRAINT fk_course_faculty FOREIGN KEY (faculty_id)
        REFERENCES faculty (faculty_id)
        ON UPDATE CASCADE ON DELETE RESTRICT
);

-- Student M : N Course (associative table)
CREATE TABLE enrollment (
    enrollment_id INT GENERATED ALWAYS AS IDENTITY,
    student_id INT NOT NULL,
```

```
course_id      INT NOT NULL,
enrollment_date DATE NOT NULL DEFAULT CURRENT_DATE,
marks          NUMERIC(5,2),
CONSTRAINT pk_enrollment PRIMARY KEY (enrollment_id),
CONSTRAINT uq_enrollment_student_course UNIQUE (student_id, course_id),
CONSTRAINT chk_enrollment_marks CHECK (marks BETWEEN 0 AND 100),
CONSTRAINT fk_enrollment_student FOREIGN KEY (student_id)
    REFERENCES student (student_id)
    ON UPDATE CASCADE ON DELETE CASCADE,
CONSTRAINT fk_enrollment_course FOREIGN KEY (course_id)
    REFERENCES course (course_id)
    ON UPDATE CASCADE ON DELETE CASCADE
);
```

15. Execution Evidence

The screenshots in this section were captured from interactive psql 17.11 terminal sessions connected to a PostgreSQL 17.11 server (Linux, local socket, port 5433), executed on 06-10-2026. Every command visible was actually executed and every result shown was produced by PostgreSQL. The images are unedited apart from cropping empty space around the terminal text.

No.	Commands executed	Purpose
1	psql --version; SHOW server_version; CREATE DATABASE college_db; \l college_db; \c college_db	Confirms PostgreSQL 17 and creates the database college_db.
2	CREATE TABLE department / student / faculty	Creates the parent table and the two tables that reference it.
3	CREATE TABLE course / enrollment; \dt	Creates the remaining tables and lists all five tables.
4	\d student; \d faculty	Shows columns, PK, UNIQUE and FK constraints of student and faculty.
5	\d course; \d enrollment	Shows columns, PK, UNIQUE, CHECK and FK constraints of course and enrollment.
6	SELECT ... FROM information_schema.table_constraints ...	Lists every primary key and foreign key with the referenced table.
7	INSERT INTO department / faculty / student / course / enrollment	Inserts sample rows to test the schema.
8	SELECT ... FROM enrollment JOIN student JOIN course JOIN faculty	Joins the tables through their foreign keys.
9	Invalid INSERTs and DELETE	Shows PostgreSQL rejecting a missing FK value, marks > 100, a duplicate enrollment and deletion of a referenced department.

```

$ psql --version
psql (PostgreSQL) 17.11
$ psql -h /tmp -p 5433 -U postgres -P pager=off
psql (17.11)
Type "help" for help.

postgres=# SHOW server_version;
server_version
-----
17.11
(1 row)

postgres=# CREATE DATABASE college_db;
CREATE DATABASE
postgres=# \l college_db
                                List of databases
  Name      | Owner   | Encoding | Locale Provider | Collate | Ctype   | Locale | ICU Rules | Access privileges
-----+-----+-----+-----+-----+-----+-----+-----+-----
 college_db | postgres | UTF8     | libc            | C.UTF-8 | C.UTF-8 |         |          |
(1 row)

postgres=# \c college_db
You are now connected to database "college_db" as user "postgres".
college_db=# █

```

Screenshot 1: Checking the PostgreSQL version, creating the database college_db and connecting to it

```

$ psql -h /tmp -p 5433 -U postgres -P pager=off -d college_db
psql (17.11)
Type "help" for help.

college_db=# CREATE TABLE department (
college_db(#   department_id  INT GENERATED ALWAYS AS IDENTITY,
college_db(#   department_name VARCHAR(100) NOT NULL,
college_db(#   CONSTRAINT pk_department PRIMARY KEY (department_id),
college_db(#   CONSTRAINT uq_department_name UNIQUE (department_name)
college_db(# );
CREATE TABLE
college_db=# CREATE TABLE student (
college_db(#   student_id    INT GENERATED ALWAYS AS IDENTITY,
college_db(#   student_name  VARCHAR(100) NOT NULL,
college_db(#   email         VARCHAR(120) NOT NULL,
college_db(#   phone         VARCHAR(15),
college_db(#   department_id INT NOT NULL,
college_db(#   CONSTRAINT pk_student PRIMARY KEY (student_id),
college_db(#   CONSTRAINT uq_student_email UNIQUE (email),
college_db(#   CONSTRAINT fk_student_department FOREIGN KEY (department_id)
college_db(#           REFERENCES department (department_id)
college_db(#           ON UPDATE CASCADE ON DELETE RESTRICT
college_db(# );
CREATE TABLE
college_db=# CREATE TABLE faculty (
college_db(#   faculty_id    INT GENERATED ALWAYS AS IDENTITY,
college_db(#   faculty_name  VARCHAR(100) NOT NULL,
college_db(#   department_id INT NOT NULL,
college_db(#   CONSTRAINT pk_faculty PRIMARY KEY (faculty_id),
college_db(#   CONSTRAINT fk_faculty_department FOREIGN KEY (department_id)
college_db(#           REFERENCES department (department_id)
college_db(#           ON UPDATE CASCADE ON DELETE RESTRICT
college_db(# );
CREATE TABLE
college_db=# █

```

Screenshot 2: Creating the tables department, student and faculty

```

$ psql -h /tmp -p 5433 -U postgres -P pager=off -d college_db
psql (17.11)
Type "help" for help.

college_db=# CREATE TABLE course (
college_db(#   course_id  INT GENERATED ALWAYS AS IDENTITY,
college_db(#   course_name VARCHAR(100) NOT NULL,
college_db(#   credits    SMALLINT NOT NULL,
college_db(#   faculty_id INT NOT NULL,
college_db(#   CONSTRAINT pk_course PRIMARY KEY (course_id),
college_db(#   CONSTRAINT chk_course_credits CHECK (credits BETWEEN 1 AND 6),
college_db(#   CONSTRAINT fk_course_faculty FOREIGN KEY (faculty_id)
college_db(#       REFERENCES faculty (faculty_id)
college_db(#       ON UPDATE CASCADE ON DELETE RESTRICT
college_db(# );
CREATE TABLE
college_db=# CREATE TABLE enrollment (
college_db(#   enrollment_id  INT GENERATED ALWAYS AS IDENTITY,
college_db(#   student_id     INT NOT NULL,
college_db(#   course_id      INT NOT NULL,
college_db(#   enrollment_date DATE NOT NULL DEFAULT CURRENT_DATE,
college_db(#   marks          NUMERIC(5,2),
college_db(#   CONSTRAINT pk_enrollment PRIMARY KEY (enrollment_id),
college_db(#   CONSTRAINT uq_enrollment_student_course UNIQUE (student_id, course_id),
college_db(#   CONSTRAINT chk_enrollment_marks CHECK (marks BETWEEN 0 AND 100),
college_db(#   CONSTRAINT fk_enrollment_student FOREIGN KEY (student_id)
college_db(#       REFERENCES student (student_id)
college_db(#       ON UPDATE CASCADE ON DELETE CASCADE,
college_db(#   CONSTRAINT fk_enrollment_course FOREIGN KEY (course_id)
college_db(#       REFERENCES course (course_id)
college_db(#       ON UPDATE CASCADE ON DELETE CASCADE
college_db(# );
CREATE TABLE
college_db=# \dt
          List of relations
Schema |   Name   | Type | Owner
-----+-----+-----+-----
public | course   | table | postgres
public | department | table | postgres
public | enrollment | table | postgres
public | faculty   | table | postgres
public | student   | table | postgres
(5 rows)

college_db=# █

```

Screenshot 3: Creating the tables course and enrollment, then listing all tables with \dt

```

$ psql -h /tmp -p 5433 -U postgres -P pager=off -d college_db
psql (17.11)
Type "help" for help.

college_db=# \d student
               Table "public.student"
   Column   |      Type      | Collation | Nullable |      Default
-----+-----+-----+-----+-----
 student_id | integer        |           | not null | generated always as identity
 student_name | character varying(100) |           | not null |
 email      | character varying(120) |           | not null |
 phone      | character varying(15)  |           |           |
 department_id | integer        |           | not null |
Indexes:
    "pk_student" PRIMARY KEY, btree (student_id)
    "uq_student_email" UNIQUE CONSTRAINT, btree (email)
Foreign-key constraints:
    "fk_student_department" FOREIGN KEY (department_id) REFERENCES department(department_id) ON UPDATE CASCADE ON DELETE RESTRICT
Referenced by:
    TABLE "enrollment" CONSTRAINT "fk_enrollment_student" FOREIGN KEY (student_id) REFERENCES student(student_id) ON UPDATE CASCADE ON DELETE CASCADE

college_db=# \d faculty
               Table "public.faculty"
   Column   |      Type      | Collation | Nullable |      Default
-----+-----+-----+-----+-----
 faculty_id | integer        |           | not null | generated always as identity
 faculty_name | character varying(100) |           | not null |
 department_id | integer        |           | not null |
Indexes:
    "pk_faculty" PRIMARY KEY, btree (faculty_id)
Foreign-key constraints:
    "fk_faculty_department" FOREIGN KEY (department_id) REFERENCES department(department_id) ON UPDATE CASCADE ON DELETE RESTRICT
Referenced by:
    TABLE "course" CONSTRAINT "fk_course_faculty" FOREIGN KEY (faculty_id) REFERENCES faculty(faculty_id) ON UPDATE CASCADE ON DELETE RESTRICT

college_db=# █

```

Screenshot 4: Structure and constraints of the student and faculty tables (\d)

```
$ psql -h /tmp -p 5433 -U postgres -P pager=off -d college_db
psql (17.11)
Type "help" for help.

college_db=# \d course
                                Table "public.course"
  Column   |      Type      | Collation | Nullable |      Default
-----+-----+-----+-----+-----
course_id | integer        |           | not null | generated always as identity
course_name | character varying(100) |           | not null |
credits    | smallint       |           | not null |
faculty_id | integer        |           | not null |
Indexes:
    "pk_course" PRIMARY KEY, btree (course_id)
Check constraints:
    "chk_course_credits" CHECK (credits >= 1 AND credits <= 6)
Foreign-key constraints:
    "fk_course_faculty" FOREIGN KEY (faculty_id) REFERENCES faculty(faculty_id) ON UPDATE CASCADE ON DELETE RESTRICT
Referenced by:
    TABLE "enrollment" CONSTRAINT "fk_enrollment_course" FOREIGN KEY (course_id) REFERENCES course(course_id) ON UPDATE CASCADE ON DELETE CASCADE

college_db=# \d enrollment
                                Table "public.enrollment"
  Column      |      Type      | Collation | Nullable |      Default
-----+-----+-----+-----+-----
enrollment_id | integer        |           | not null | generated always as identity
student_id    | integer        |           | not null |
course_id     | integer        |           | not null |
enrollment_date | date           |           | not null | CURRENT_DATE
marks         | numeric(5,2)   |           |           |
Indexes:
    "pk_enrollment" PRIMARY KEY, btree (enrollment_id)
    "uq_enrollment_student_course" UNIQUE CONSTRAINT, btree (student_id, course_id)
Check constraints:
    "chk_enrollment_marks" CHECK (marks >= 0::numeric AND marks <= 100::numeric)
Foreign-key constraints:
    "fk_enrollment_course" FOREIGN KEY (course_id) REFERENCES course(course_id) ON UPDATE CASCADE ON DELETE CASCADE
    "fk_enrollment_student" FOREIGN KEY (student_id) REFERENCES student(student_id) ON UPDATE CASCADE ON DELETE CASCADE

college_db=# █
```

Screenshot 5: Structure and constraints of the course and enrollment tables (\d)

```
$ psql -h /tmp -p 5433 -U postgres -P pager=off -d college_db
psql (17.11)
Type "help" for help.

college_db=# SELECT tc.table_name, tc.constraint_name, tc.constraint_type,
college_db=#         kcu.column_name, ccu.table_name AS references_table
college_db=# FROM information_schema.table_constraints tc
college_db=# JOIN information_schema.key_column_usage kcu
college_db=# ON tc.constraint_name = kcu.constraint_name
college_db=# LEFT JOIN information_schema.constraint_column_usage ccu
college_db=# ON tc.constraint_name = ccu.constraint_name
college_db=# AND tc.constraint_type = 'FOREIGN KEY'
college_db=# WHERE tc.table_schema = 'public'
college_db=# AND tc.constraint_type IN ('PRIMARY KEY', 'FOREIGN KEY')
college_db=# ORDER BY tc.table_name, tc.constraint_type DESC;
 table_name | constraint_name | constraint_type | column_name | references_table
-----+-----+-----+-----+-----
course     | pk_course       | PRIMARY KEY    | course_id   |
course     | fk_course_faculty | FOREIGN KEY    | faculty_id  | faculty
department | pk_department   | PRIMARY KEY    | department_id |
enrollment | pk_enrollment   | PRIMARY KEY    | enrollment_id |
enrollment | fk_enrollment_student | FOREIGN KEY    | student_id  | student
enrollment | fk_enrollment_course | FOREIGN KEY    | course_id   | course
faculty     | pk_faculty      | PRIMARY KEY    | faculty_id  |
faculty     | fk_faculty_department | FOREIGN KEY    | department_id | department
student     | pk_student      | PRIMARY KEY    | student_id  |
student     | fk_student_department | FOREIGN KEY    | department_id | department
(10 rows)

college_db=# █
```

Screenshot 6: Verifying all primary keys and foreign keys from information_schema

```

$ psql -h /tmp -p 5433 -U postgres -P pager=off -d college_db
psql (17.11)
Type "help" for help.

college_db=# INSERT INTO department (department_name) VALUES
college_db=#      ('Computer Science and Engineering'),
college_db=#      ('Artificial Intelligence and Machine Learning'),
college_db=#      ('Electronics and Telecommunication');
INSERT 0 3
college_db=# INSERT INTO faculty (faculty_name, department_id) VALUES
college_db=#      ('Dr. Anita Kulkarni', 1),
college_db=#      ('Prof. Rahul Mehta', 2),
college_db=#      ('Dr. Sunil Patil', 3);
INSERT 0 3
college_db=# INSERT INTO student (student_name, email, phone, department_id) VALUES
college_db=#      ('Aarav Sharma', 'aarav.sharma@college.edu', '9876500001', 1),
college_db=#      ('Priya Nair', 'priya.nair@college.edu', '9876500002', 2),
college_db=#      ('Rohan Desai', 'rohan.desai@college.edu', '9876500003', 2),
college_db=#      ('Sneha Joshi', 'sneha.joshi@college.edu', '9876500004', 3);
INSERT 0 4
college_db=# INSERT INTO course (course_name, credits, faculty_id) VALUES
college_db=#      ('Database Management Systems', 4, 1),
college_db=#      ('Machine Learning', 3, 2),
college_db=#      ('Digital Electronics', 3, 3);
INSERT 0 3
college_db=# INSERT INTO enrollment (student_id, course_id, enrollment_date, marks) VALUES
college_db=#      (1, 1, '2026-07-15', 82.50),
college_db=#      (2, 1, '2026-07-15', 91.00),
college_db=#      (2, 2, '2026-07-16', 88.00),
college_db=#      (3, 2, '2026-07-16', 76.50),
college_db=#      (4, 3, '2026-07-17', 69.00),
college_db=#      (3, 1, '2026-07-17', NULL);
INSERT 0 6
college_db=# █

```

Screenshot 7: Inserting sample data into all five tables

```

$ psql -h /tmp -p 5433 -U postgres -P pager=off -d college_db
psql (17.11)
Type "help" for help.

college_db=# SELECT s.student_name, c.course_name, f.faculty_name,
college_db=#      e.enrollment_date, e.marks
college_db=# FROM enrollment e
college_db=# JOIN student s ON s.student_id = e.student_id
college_db=# JOIN course c ON c.course_id = e.course_id
college_db=# JOIN faculty f ON f.faculty_id = c.faculty_id
college_db=# ORDER BY e.enrollment_id;

```

student_name	course_name	faculty_name	enrollment_date	marks
Aarav Sharma	Database Management Systems	Dr. Anita Kulkarni	2026-07-15	82.50
Priya Nair	Database Management Systems	Dr. Anita Kulkarni	2026-07-15	91.00
Priya Nair	Machine Learning	Prof. Rahul Mehta	2026-07-16	88.00
Rohan Desai	Machine Learning	Prof. Rahul Mehta	2026-07-16	76.50
Sneha Joshi	Digital Electronics	Dr. Sunil Patil	2026-07-17	69.00
Rohan Desai	Database Management Systems	Dr. Anita Kulkarni	2026-07-17	

```

(6 rows)

college_db=# █

```

Screenshot 8: Join query across enrollment, student, course and faculty

```
$ psql -h /tmp -p 5433 -U postgres -P pager=off -d college_db
psql (17.11)
Type "help" for help.

college_db=# INSERT INTO enrollment (student_id, course_id) VALUES (99, 1);
ERROR: insert or update on table "enrollment" violates foreign key constraint "fk_enrollment_student"
DETAIL: Key (student_id)=(99) is not present in table "student".
college_db=# INSERT INTO enrollment (student_id, course_id, marks) VALUES (4, 1, 120);
ERROR: new row for relation "enrollment" violates check constraint "chk_enrollment_marks"
DETAIL: Failing row contains (8, 4, 1, 2026-10-06, 120.00).
college_db=# INSERT INTO enrollment (student_id, course_id) VALUES (1, 1);
ERROR: duplicate key value violates unique constraint "uq_enrollment_student_course"
DETAIL: Key (student_id, course_id)=(1, 1) already exists.
college_db=# DELETE FROM department WHERE department_id = 1;
ERROR: update or delete on table "department" violates foreign key constraint "fk_student_department" on table "student"
DETAIL: Key (department_id)=(1) is still referenced from table "student".
college_db=# █
```

Screenshot 9: Constraint enforcement – invalid operations rejected by PostgreSQL

16. Result and Conclusion

An ER model for the College Management System was designed with five entities – Department, Student, Faculty, Course and Enrollment – and four relationships: Department 1 : N Student, Department 1 : N Faculty, Faculty 1 : N Course, and Student M : N Course resolved through the Enrollment associative entity. The ER diagram was mapped into a relational schema using standard mapping rules, and the database college_db with all five tables was successfully created in PostgreSQL 17.

The execution output confirms that all primary key, foreign key, UNIQUE and CHECK constraints were created as designed, that data can be joined correctly across all five tables, and that PostgreSQL rejects invalid data (non-existent student, marks above 100, duplicate enrollment, deletion of a referenced department). Hence, the practical was performed successfully.

Date: _____

Signature of Faculty: _____

17. Viva Questions and Answers

Q1. What is an ER model?

The Entity–Relationship model is a high-level conceptual data model that describes the data of a system in terms of entities, their attributes and the relationships among them. It is drawn as an ER diagram and later converted into tables.

Q2. What is the difference between an entity and an entity set?

An entity is a single real-world object that can be distinctly identified, e.g. one particular student. An entity set is the collection of all entities of the same type, e.g. all students; it becomes a table in the relational model.

Q3. What are the different types of attributes?

Simple (atomic) and composite, single-valued and multi-valued, stored and derived, and key attributes. A null-able attribute such as *marks* may also have no value.

Q4. Define super key, candidate key and primary key.

A super key is any set of attributes that uniquely identifies a row. A candidate key is a minimal super key. The primary key is the candidate key chosen by the designer to identify rows; it cannot be NULL. In *student*, both *student_id* and *email* are candidate keys and *student_id* is the primary key.

Q5. What is a foreign key and what is referential integrity?

A foreign key is an attribute in one table that refers to the primary key of another table. Referential integrity means every non-null foreign key value must match an existing primary key value in the referenced table, e.g. *student.department_id* must exist in *department*.

Q6. What is cardinality? Name its types.

Cardinality (mapping cardinality) is the number of entities of one set that can be associated with entities of another set through a relationship: one-to-one (1:1), one-to-many (1:N), many-to-one (N:1) and many-to-many (M:N).

Q7. What is the difference between total and partial participation?

In total participation every entity of the set must take part in the relationship (shown by a double line), e.g. every student must belong to a department. In partial participation some entities may not take part, e.g. a department may have no students yet.

Q8. How is a 1 : N relationship mapped to tables?

The primary key of the table on the “1” side is placed as a foreign key in the table on the “N” side. For *Department 1 : N Student*, *department_id* is added to the *student* table.

Q9. How is an M : N relationship mapped, and why is the Enrollment table needed?

An M : N relationship cannot be represented by a single foreign key, so a new table is created containing the primary keys of both entities as foreign keys plus any relationship attributes. Enrollment stores *student_id*, *course_id*, *enrollment_date* and *marks*, converting *Student M : N Course* into two 1 : N relationships.

Q10. What is an associative entity? Is Enrollment a weak entity?

An associative entity is an entity created to represent an M : N relationship that has its own attributes. Enrollment is not a weak entity here because it has its own primary key (enrollment_id) and does not depend on an owner entity for identification.

Q11. Why is UNIQUE (student_id, course_id) used in the enrollment table?

Because enrollment_id is a surrogate key, it would not by itself prevent the same student from being enrolled in the same course twice. The UNIQUE constraint enforces this business rule and makes (student_id, course_id) an alternate key.

Q12. What does GENERATED ALWAYS AS IDENTITY do?

It makes PostgreSQL automatically generate a unique sequential value for the column on every insert. It is the SQL-standard identity column and is preferred over the older SERIAL pseudo-type. With ALWAYS, users cannot insert their own value unless they use OVERRIDING SYSTEM VALUE.

Q13. What is the difference between ON DELETE CASCADE and ON DELETE RESTRICT?

CASCADE automatically deletes the child rows when the parent row is deleted (deleting a student removes the student's enrollments). RESTRICT refuses to delete a parent row while child rows still refer to it (a department with students cannot be deleted), as shown in Screenshot 9.

Q14. What are DDL and DML? Give examples.

DDL (Data Definition Language) defines the structure of the database: CREATE, ALTER, DROP, TRUNCATE. DML (Data Manipulation Language) works on the data: INSERT, UPDATE, DELETE, SELECT.

Q15. What is a relational schema? Define degree and cardinality of a relation.

A relational schema is the name of a relation together with its list of attributes, e.g. COURSE(course_id, course_name, credits, faculty_id). The degree is the number of attributes (4 for course) and the cardinality is the number of tuples (rows) in the relation at a given time.